

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance, using a single CNN architecture (MobileNetV2) on the Oxford-IIIT Pet dataset.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset was loaded through TensorFlow Datasets (`tfds`), which gave a training set of 2,944 images, a validation set of 736 images and an independent test set of 3,669 images across all 37 breeds. The test set was kept completely untouched until the very last section of this experiment.

Experimental considerations:

- Used RGB images resized to 224×224 .
- Normalized the input images according to MobileNetV2's own preprocessing requirements.
- Maintained separate training, validation and test data throughout.
- The test set was not looked at during any of the weight initialization, regularization, optimizer, hyperparameter or cross-validation stages.
- MobileNetV2 was used because it is computationally lighter than large CNN architectures such as ResNet or VGG, which made repeated re-training across all the studies below feasible on a single Kaggle GPU session.

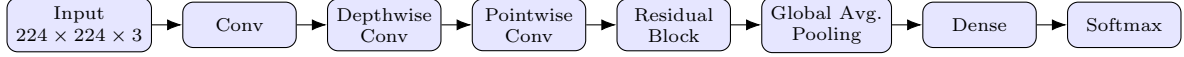
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

A baseline MobileNetV2 model (ImageNet-pretrained base, frozen, with a new classification head) was trained for 5 epochs before any of the studies below, purely to have a reference point:

Metric	Baseline Value
Validation Accuracy	90.63%
Validation Loss	0.2937
Training Time	51.02 s
Trainable Parameters	2,305,381

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Four initialization strategies were compared, all on the same frozen MobileNetV2 base with only the new classifier head's weights initialized differently:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Initialization	Final Training Loss	Best Validation Accuracy (%)	Training Time (s)
Zero	0.0905	91.30	41.19
Random	0.1023	90.76	41.08
Xavier/Glorot	0.0985	90.49	40.34
He	0.0968	90.62	45.07

Selected best initialization: Zero.

Plot 1: Training Loss vs. Epoch

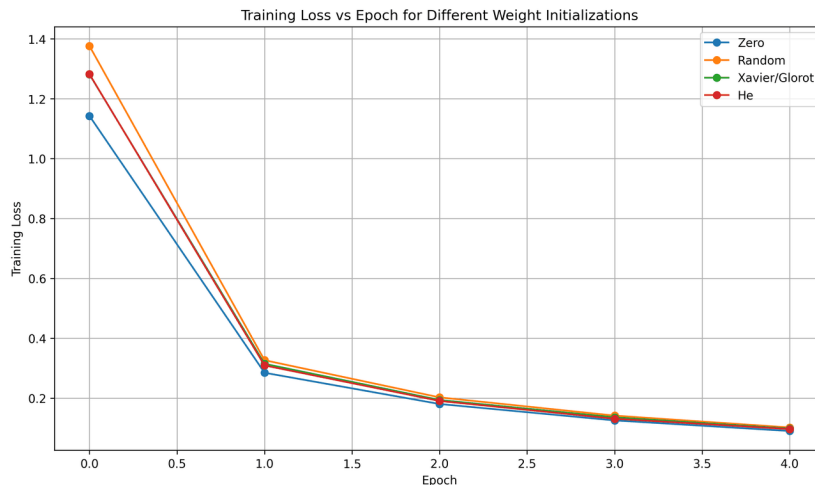


Figure 1: Training loss vs. epoch for the four weight initialization strategies.

Inference: All four initializations converge to a low training loss within 5 epochs since only the classifier head is trained on a frozen base. Zero initialization stays marginally lowest throughout – the usual symmetry problem with zero init mainly hurts networks trained from scratch, not a single head on frozen features.

Plot 2: Validation Accuracy vs. Epoch

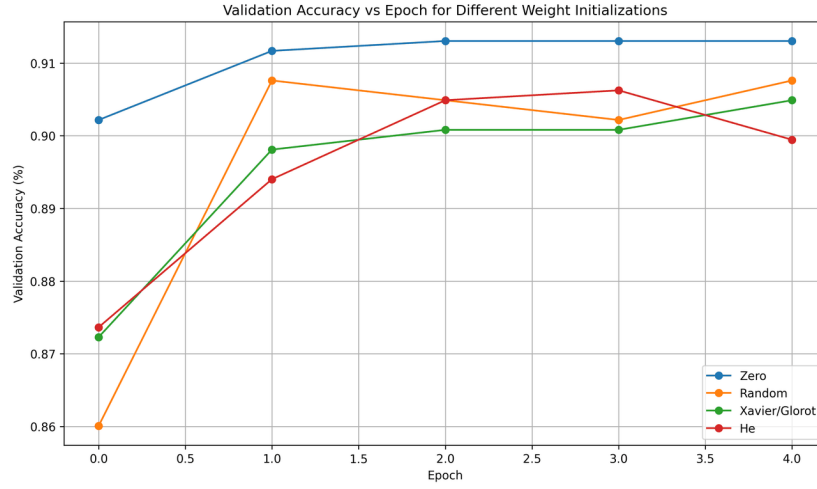


Figure 2: Validation accuracy vs. epoch for the four weight initialization strategies.

Inference: Zero initialization reaches the highest, most stable validation accuracy (91.3%) fastest, while Random starts lowest (86%) but recovers. All four end up within 1.5% of each other, showing that head initialization matters far less with a frozen, pretrained backbone than it would training from scratch.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Four regularization settings were compared, all using Zero initialization (the best setting from Section 5):

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Regularization	Final Training Loss	Best Validation Accuracy (%)	Training Time (s)
No Regularization	0.0335	90.63	40.96
L2 Regularization	0.2423	90.35	40.98
Dropout	0.2957	91.17	42.01
Batch Normalization	0.0358	91.30	41.86

Plot 3: Training and Validation Accuracy vs. Epoch

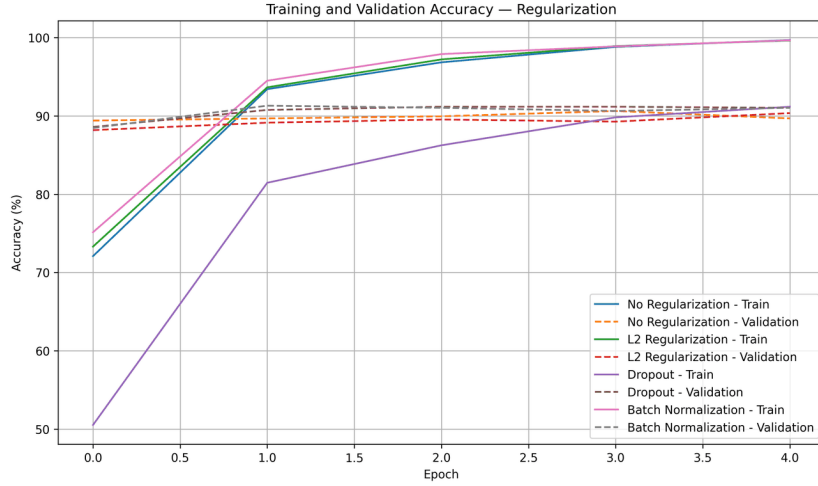


Figure 3: Training and validation accuracy across epochs for each regularization setting.

Inference: “No Regularization” and “Batch Normalization” shoot up close to 100% training accuracy while validation plateaus at 90–91%, showing a clear generalization gap. Dropout’s training curve climbs slowest but tracks its own validation curve most closely – the hallmark of a working regularizer.

Plot 4: Training and Validation Loss vs. Epoch

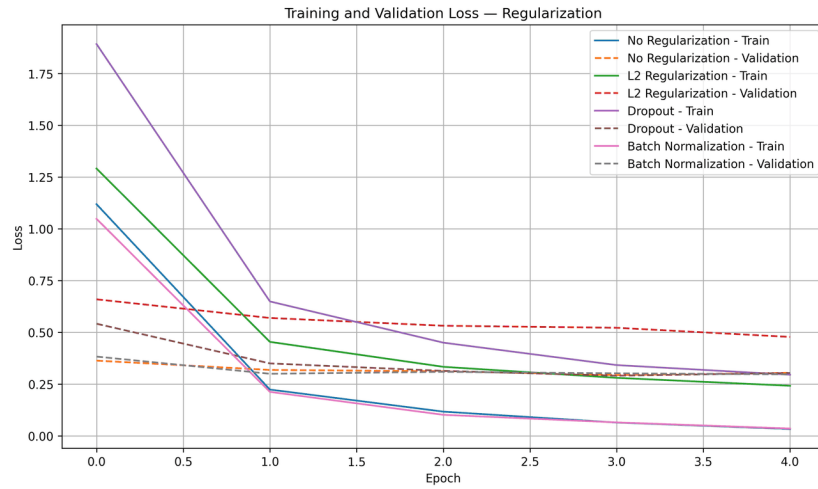


Figure 4: Training and validation loss across epochs for each regularization setting.

Inference: Validation loss for “No Regularization” and “Batch Normalization” flattens early while training loss keeps falling – the classic overfitting signature. Dropout’s validation and training loss stay closest together, confirming it trades some raw accuracy for a smaller generalization gap.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift, so normalization never permanently removes any representational power – the network can always learn to undo it if that is optimal.

Plot 5: With vs. Without Batch Normalization

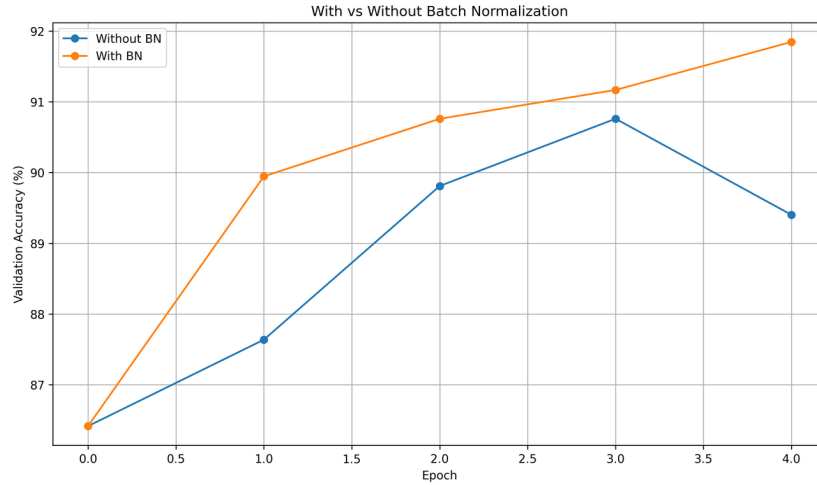


Figure 5: Validation accuracy with and without Batch Normalization.

Inference: Batch Normalization consistently pushes validation accuracy higher, finishing at 91.85% vs. 89.40% without it, and the curve is smoother – the non-BN curve even dips at the final epoch. Normalizing activations keeps the head’s inputs stable batch to batch.

8. Optimization Algorithms

Four optimizers were compared, all with Zero initialization and no extra regularization, to isolate the effect of the optimizer itself:

- SGD
- Momentum
- RMSProp
- Adam

Plot 6: Training Loss vs. Epoch for Different Optimizers

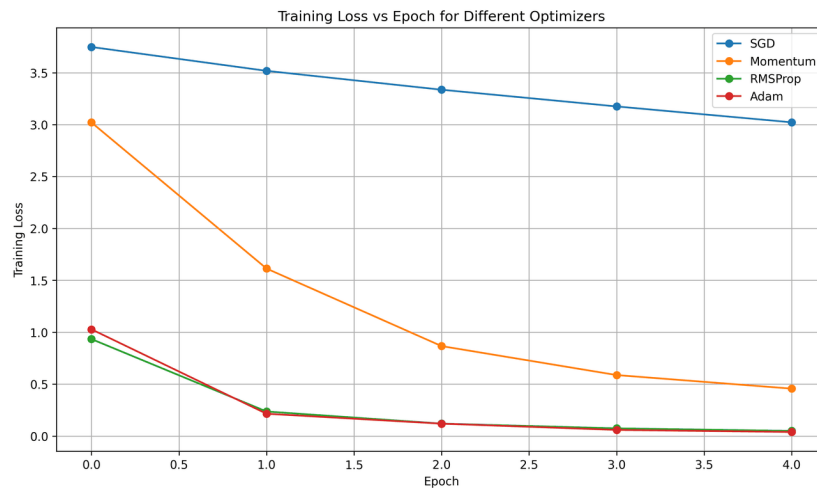


Figure 6: Training loss vs. epoch for SGD, Momentum, RMSProp and Adam.

Inference: RMSProp and Adam collapse training loss to under 0.05 within 5 epochs thanks to per-parameter adaptive steps. Momentum lags (above 0.4), and plain SGD barely moves (ending near 3.0) – a fixed learning rate with no adaptivity is too slow for this short a budget.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

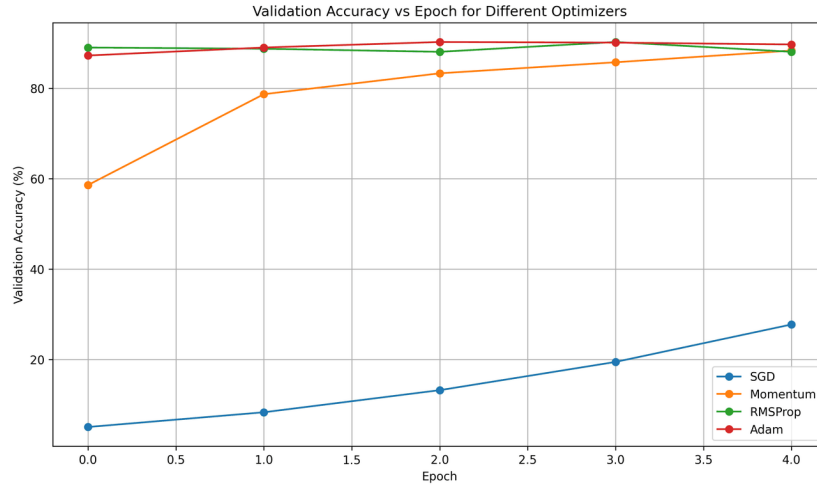


Figure 7: Validation accuracy vs. epoch for SGD, Momentum, RMSProp and Adam.

Inference: Adam and RMSProp reach ~90% validation accuracy almost immediately (Adam in just 3 epochs). Momentum climbs steadily to 86%, while SGD crawls to only 28% – confirming a fixed learning rate alone is a poor fit for this short a budget.

Optimizer	Final Loss	Best Val. Accuracy (%)	Epoch to Converge	Time (s)
SGD	3.0220	27.72	5	39.93
Momentum	0.4563	88.32	5	40.80
RMSProp	0.0491	90.22	4	40.49
Adam	0.0384	90.22	3	40.64

9. CNN Hyperparameter Tuning

The following hyperparameters were investigated one at a time, keeping all other settings fixed at their best value from the earlier sections:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

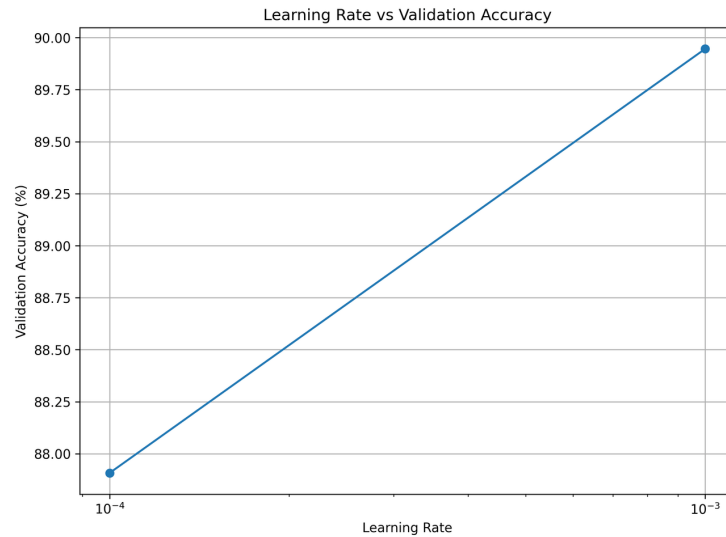


Figure 8: Validation accuracy for learning rates 10^{-3} and 10^{-4} .

Inference: 0.001 reaches higher accuracy (89.95%) than 0.0001 (87.91%) in the same 5 epochs – simply a budget effect, since smaller steps cover less ground in a fixed number of epochs.

Plot 9: Batch Size vs. Validation Accuracy

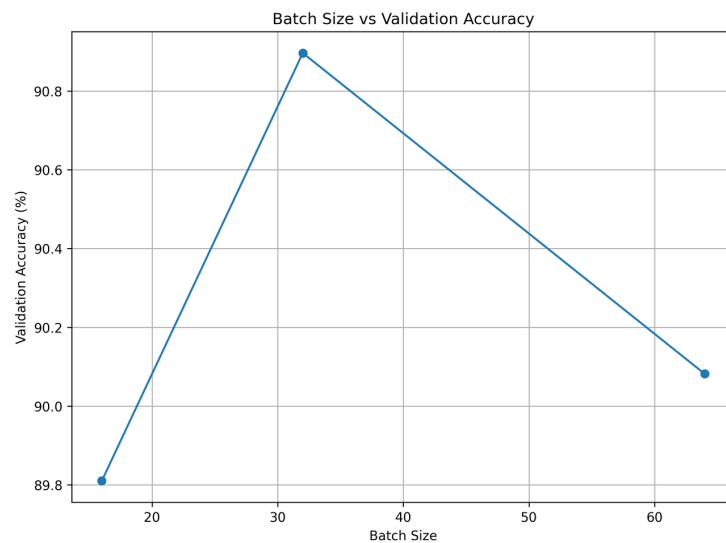


Figure 9: Validation accuracy for batch sizes 16, 32 and 64.

Inference: Batch size 32 gives the best accuracy (90.90%), with 64 close behind and 16 lowest. Small batches give noisier gradients that slow convergence in only 5 epochs; 32 balances stability and update count well.

Plot 10: Dropout Rate vs. Validation Accuracy

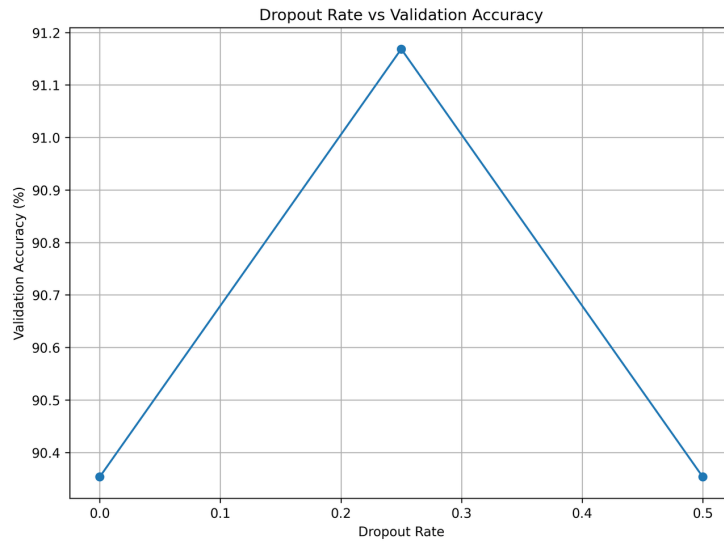


Figure 10: Validation accuracy for dropout rates 0, 0.25 and 0.5.

Inference: A moderate 0.25 dropout gives the best accuracy (91.17%), with 0 and 0.5 tied lower (90.35%) – the classic trade-off, where too little under-regularizes and too much slows learning in 5 epochs.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet was used in two modes.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate (six trainable layers/blocks were unfrozen in this run).

Configuration	Best Val. Accuracy (%)	Final Val. Loss	Training Time (s)	Parameters
Feature Extraction	89.95	0.3069	41.96	2,426,725
Fine-Tuning	82.20	0.8322	45.18	2,426,725

Plot 11: Feature Extraction vs. Fine-Tuning

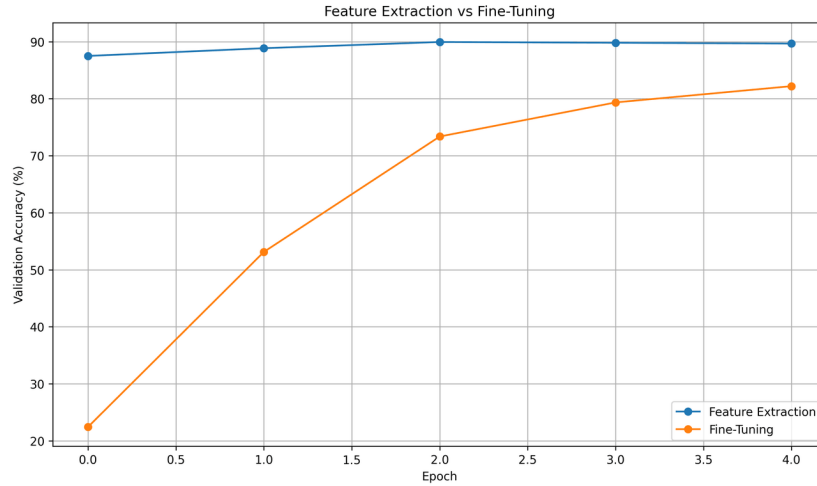


Figure 11: Validation accuracy: feature extraction vs. fine-tuning, across epochs.

Inference: Feature Extraction starts strong (87.5% at epoch 1) and stays near 90%. Fine-Tuning starts near scratch (22%) since unfreezing disrupts pretrained features, reaching only 82% by epoch 5 – Feature Extraction is clearly more epoch-efficient here.

Plot 12: Training and Validation Loss

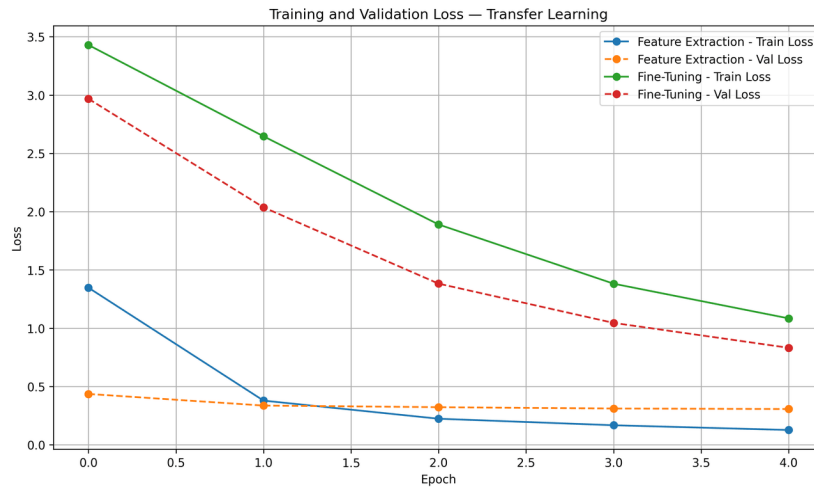


Figure 12: Training and validation loss for feature extraction and fine-tuning.

Inference: Feature Extraction’s train/val loss converge close together by epoch 2 – a well-fit model. Fine-Tuning’s loss starts much higher (3.4) and is still above 1.0 by epoch 5, with no overfitting yet – just more learning left to do.

Discussion: A large learning rate on the unfrozen, already pretrained layers would erase their useful weights in a few steps – undoing the point of transfer learning. A small rate lets them adapt gently instead.

11. K-Fold Cross-Validation

Based on the individual studies above, four promising configurations were selected for 5-fold cross-validation on the training data:

- **C1 – Feature Extraction:** frozen MobileNetV2 base, plain classifier head.
- **C2 – Dropout 0.25:** frozen base, classifier head with 0.25 dropout (best dropout rate from Section 9).
- **C3 – Batch Normalization:** frozen base, classifier head with a BatchNorm layer (best regularization from Section 6).
- **C4 – Fine-Tuning:** MobileNetV2 with the top layers unfrozen and a small fine-tuning learning rate.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set was not used at any point during this selection.

Configuration	F1 (%)	F2 (%)	F3 (%)	F4 (%)	F5 (%)	Mean $\pm$ SD (%)	Time (s)
C1 – Feature Extraction	91.51	90.15	91.17	90.15	93.20	91.24 ± 1.12	39.55
C2 – Dropout 0.25	90.49	90.49	90.49	91.51	92.35	91.07 ± 0.75	34.96
C3 – Batch Normalization	91.51	90.83	91.85	90.49	91.67	91.27 ± 0.52	34.62
C4 – Fine-Tuning	79.12	76.06	75.21	79.63	78.06	77.62 ± 1.71	39.04

Selected configuration: C3 – Batch Normalization, with the highest mean accuracy (91.27%) and, just as importantly, the lowest standard deviation (0.52%) of all four configurations.

Plot 13: 5-Fold Cross-Validation Accuracy

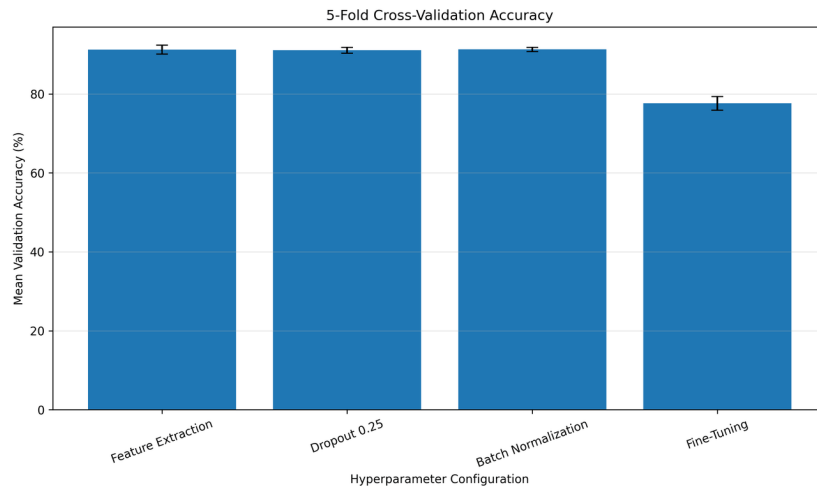


Figure 13: Mean 5-fold cross-validation accuracy with standard-deviation error bars for C1–C4.

Inference: C1, C2, C3 are tied on mean accuracy ($\sim 91\%$), but C3 has the smallest error bar – the most consistent across folds. C4 is both lower and more variable, confirming 5 epochs isn’t enough for fine-tuning to compete yet.

12. Final Model Evaluation

The selected configuration (C3 – Batch Normalization) was retrained on the complete training data, and evaluated once on the untouched test set.

Metric	Value
Mean CV Accuracy	91.27%
CV Standard Deviation	0.52%
Test Accuracy	89.10%
Precision (macro)	89.18%
Recall (macro)	89.03%
F1-score (macro)	88.95%
Training Time	41.48 s
Number of Parameters	2,431,845

Plot 13b: Final Model Training and Validation Accuracy

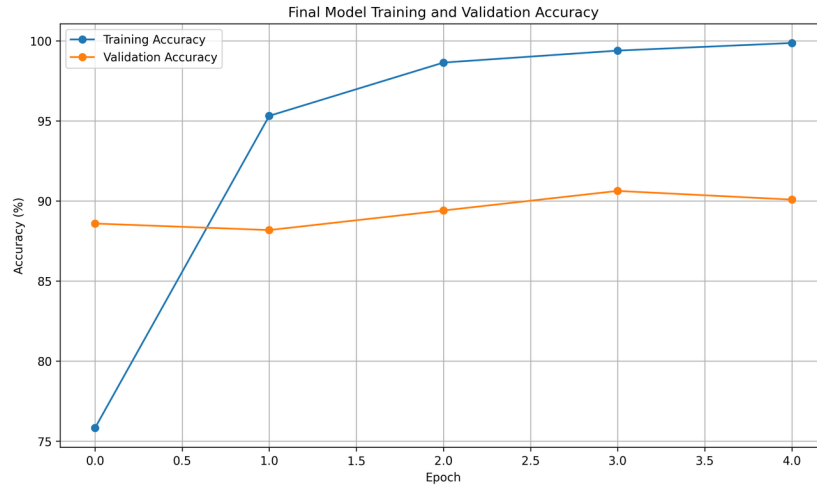


Figure 14: Training and validation accuracy while retraining the final selected model on the full training set.

Inference: Training accuracy climbs to $\sim 100\%$ by epoch 4 while validation settles at 90–91%, matching the small gap already seen for this configuration in cross-validation.

Plot 14: Confusion Matrix

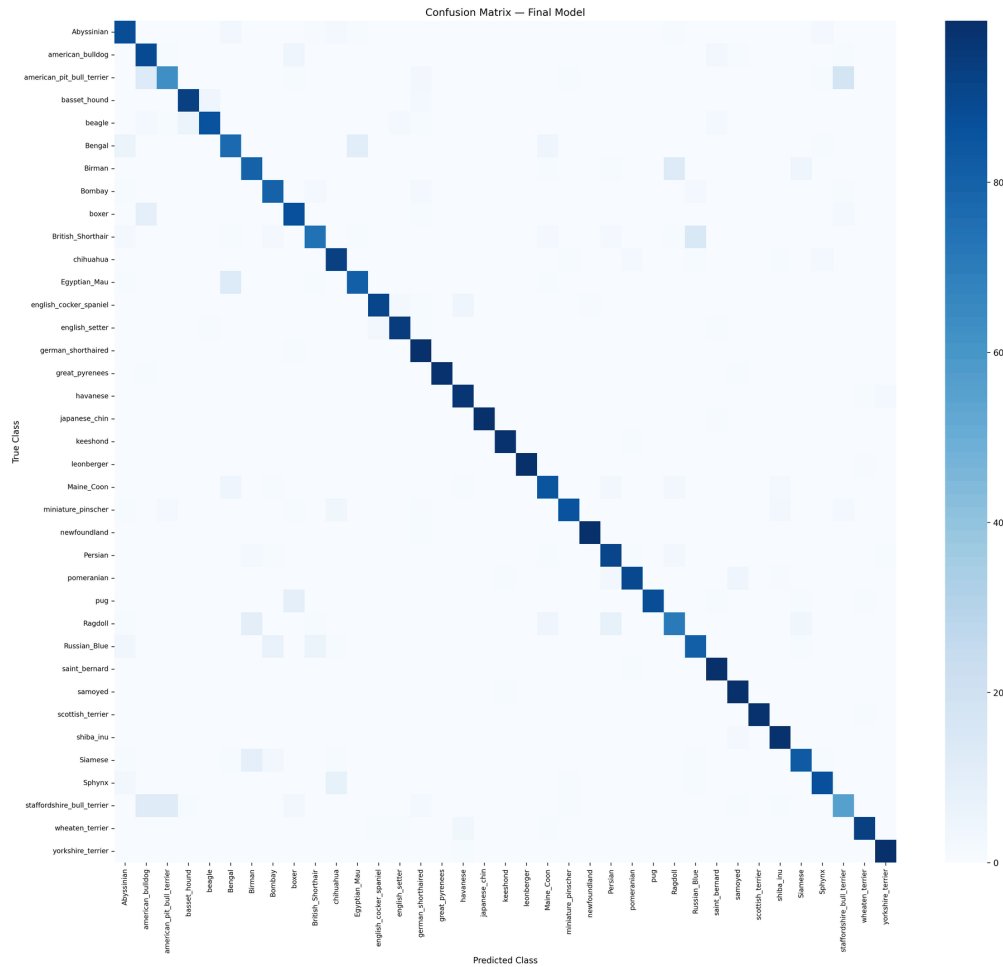


Figure 15: Confusion matrix of the final model on the independent test set, across all 37 breeds.

Inference: The matrix is overwhelmingly diagonal, matching 89.1% test accuracy. The clearest confusion is staffordshire_bull_terrier vs. american_bulldog vs. american_pit_bull_terrier – visually near-identical breeds. A fainter pattern also appears among short-haired cats (British.Shorthair, Russian.Blue, Persian, Ragdoll).

Optional Plot 15: Additional Exercise Comparison

The additional exercise asked for two new hyperparameter combinations to be evaluated with 5-fold CV and compared against the selected configuration:

- **Additional C1:** learning rate = 10^{-4} , dropout = 0.5, batch size = 16, no fine-tuning.
- **Additional C2:** learning rate = 10^{-5} , dropout = 0.5, batch size = 64, with fine-tuning.

Configuration	Mean CV Accuracy (%)	SD (%)	Mean Training Time (s)
Batch Normalization (selected)	91.27	0.52	34.62
Additional C1	87.36	1.02	39.58
Additional C2	53.43	4.27	42.52

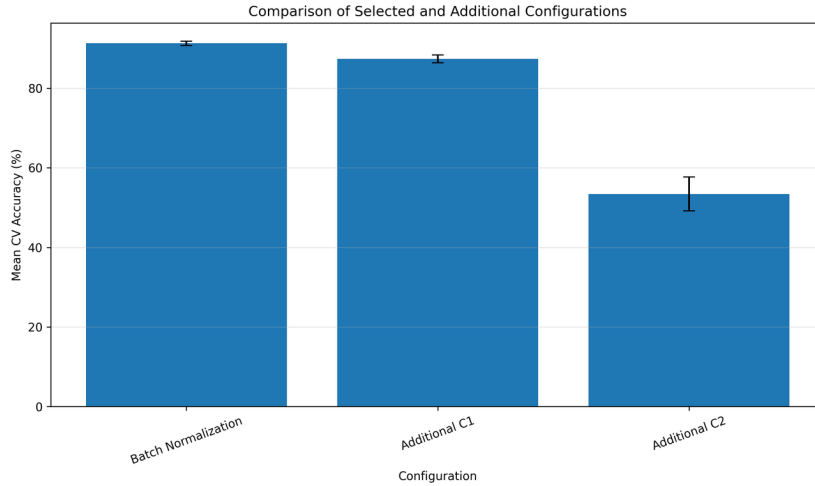


Figure 16: Comparison of the selected configuration against the two additional exercise configurations.

Inference: Neither additional configuration beats Batch Normalization. C1’s lower LR and higher dropout cost ~ 4 points and nearly double the SD. C2 is far worse (53.4%, SD 4.27%) since its tiny LR (10^{-5}) combined with fine-tuning barely moves the model in 5 epochs. **Batch Normalization remains preferable** on accuracy, variability, and cost.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time (s)
Baseline	—	—	—	51.02
Best Initialization (Zero)	—	—	—	41.19
Best Regularization (Batch Norm.)	—	—	—	41.86
Best Optimizer (RMSProp/Adam)	—	—	—	40.49 / 40.64
Best Hyperparameters (LR 0.001, BS 32, Dropout 0.25)	—	—	—	42.74
Fine-Tuned Model (C4)	77.62%	1.71%	—	39.04
Final Selected Model (C3 – Batch Norm.)	91.27%	0.52%	89.10%	41.48

Note: only C1–C4 were cross-validated, and only the final selected model was tested, per the rule that the test set is not touched during selection – so earlier stages show validation-only numbers, with CV/Test left as — rather than fabricated.

Justification: Batch Normalization (C3) had the highest mean CV accuracy (91.27%) *and* the lowest SD (0.52%), so its performance is trustworthy, not a lucky split. It also trained fastest per fold (34.62s) and generalized well to the test set (89.10%, balanced precision/recall) – close to its CV estimate.

14. Required Inference for Plots

Inferences for every plot (Plots 1–15) are provided directly beneath each figure in Sections 5–12 above, each covering what the plot shows, the trend observed, and why that trend likely occurs.

15. Discussion Questions

- What is the difference between model parameters and hyperparameters?** Parameters (weights, biases) are learned automatically during training. Hyperparameters (learning rate, batch size, dropout, epochs) are set *before* training and are not updated by backpropagation.
- Why is weight initialization important?** It determines the initial gradient signal. Poor initialization can cause activations/gradients to vanish or explode across layers; good initialization keeps signal variance stable layer to layer.
- Why can zero initialization be problematic for neural networks?** Every neuron would compute the same output and get the same gradient, so they update identically forever, never breaking symmetry. It didn’t hurt us here since only a head is trained on frozen features, but it would cripple a network trained from scratch.

4. **Compare Xavier and He initialization.** Both scale initial weights by fan-in/out to keep variance stable. Xavier assumes linear/tanh activations ($\sqrt{2/(n_{in} + n_{out})}$); He targets ReLU, using a larger variance ($\sqrt{2/n}$) since ReLU zeroes about half its inputs.
5. **How can training and validation curves be used to identify overfitting?** Overfitting shows as a growing gap: training accuracy/loss keeps improving while validation plateaus or worsens – seen for “No Regularization” and “Batch Normalization” in Plot 3/4.
6. **How does Dropout reduce overfitting?** It randomly switches off a fraction of neurons each pass, so no unit can be relied on. This behaves like training an ensemble of thinned sub-networks, reducing memorization of training-set noise.
7. **What is the purpose of Batch Normalization?** It normalizes each layer’s activations to zero mean/unit variance per mini-batch, then applies a learnable scale and shift. This stabilizes the input distribution during training, allowing faster training and higher learning rates.
8. **Explain the numerical Batch Normalization example.** For $x = [2, 4, 6, 8]$: $\mu_B = 5$, $\sigma_B^2 = 5$. Normalizing gives $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$ (zero mean, unit variance). With $\gamma = 1, \beta = 0$ these are also the final outputs.
9. **What are the roles of γ and β ?** They let the network undo or adjust normalization if beneficial: γ rescales, β shifts, so the layer isn’t forced to always output zero-mean, unit-variance activations.
10. **Compare SGD, Momentum, RMSProp and Adam.** SGD uses only the current gradient (slowest here, 27.7%). Momentum smooths updates with past gradients (88.3%). RMSProp and Adam use per-parameter adaptive rates and converged fastest (90.2% each); Adam’s added momentum term got it there in 3 epochs vs. RMSProp’s 4.
11. **What happens when the learning rate is too large?** Updates overshoot the loss minimum, causing oscillation or divergence instead of smooth convergence.
12. **What happens when the learning rate is too small?** Training becomes very slow – within a fixed epoch budget the model may not reach a good solution, as seen with LR 0.0001 in Plot 8.
13. **What is the effect of increasing batch size?** Larger batches give a less noisy gradient estimate but fewer updates per epoch. Batch size 32 outperformed both 16 (too noisy) and 64 (fewer update steps).
14. **Explain stride and padding.** Stride is how many pixels the kernel moves per step (stride 2 roughly halves output size). Padding adds border pixels so the kernel can reach edges, controlling how much the output shrinks.
15. **Why is MobileNetV2 computationally efficient?** It uses depthwise separable convolutions (depthwise + pointwise 1×1), needing far fewer multiply-adds than standard convolution, plus inverted residual blocks with linear bottlenecks to keep channel counts low where computation is heaviest.
16. **What is depthwise separable convolution?** It splits convolution into a depthwise step (one filter per channel, spatial only) and a pointwise 1×1 step (mixes channels), together approximating standard convolution far more cheaply.
17. **What is transfer learning?** It means reusing a model already trained on one dataset (MobileNetV2 on ImageNet) for a new task instead of training from scratch, since low/mid-level visual features transfer well across image tasks.
18. **Differentiate feature extraction and fine-tuning.** Feature extraction freezes the base entirely and trains only a new head. Fine-tuning unfreezes some upper layers too, adapting them at a small learning rate – more computation and, as our results showed, a longer road to convergence.
19. **Why is a smaller learning rate generally used during fine-tuning?** The unfrozen layers already hold useful pretrained weights; a large learning rate would destroy them before the new head learns anything useful. A small rate lets them adjust gradually.
20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?** A single split can be misleadingly good or bad by chance. K-Fold CV averages over K splits, giving a more reliable estimate plus, via the SD, a sense of variability.
21. **Why must the test set remain untouched during tuning?** If the test set influences any tuning decision, reported accuracy becomes optimistically biased. Keeping it untouched until Section 12 ensures the 89.10% test accuracy is a fair estimate.
22. **Why should mean and standard deviation both be reported?** The mean alone hides consistency across splits – C1, C2, C3 all had similar means ($\sim 91\%$) but different SDs; C3’s smallest SD (0.52%) made it the more dependable choice.
23. **Is the highest validation accuracy always sufficient to select a model?** No – a single high accuracy could be a lucky split and ignores consistency/cost. C1 and C3 had near-identical means, but C3 was preferred for its lower variability.

16. Additional Exercise

Two new combinations of learning rate, dropout, batch size and fine-tuning strategy were evaluated with 5-fold cross-validation and compared against the selected configuration; see Section 12 (“Optional Plot 15”) above for the

full results table, the comparison plot, and the discussion of why the selected Batch Normalization configuration remains preferable on accuracy, variability and computational cost.

17. Expected Outcome

This experiment demonstrated that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices, even with a fixed architecture and dataset. Zero initialization, Batch Normalization, and RMSProp/Adam each measurably outperformed their alternatives, and frozen-base feature extraction beat fine-tuning within the 5-epoch budget. The final configuration, chosen via 5-fold CV, generalized well to the test set (89.10%).

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. TensorFlow Documentation: <https://www.tensorflow.org>
3. Keras Documentation: <https://keras.io>